

SESIÓN 6

Migraciones y Seeds en el Pipeline; Gestión de Secretos

Informe de ejecución de los tres laboratorios

Estudiante	Albert Omar Araya Cory
Repositorio	github.com/albrtaraya/fullstack-lab-1-1ci-cd
Pull Request	#9 — integrado a main (867e3cd)
Secrets creados	DATABASE_URL · JWT_SECRET · API_KEY
Pipeline	4 jobs, con PostgreSQL efímero
Modalidad	Individual
Fecha	16 de septiembre de 2026

Insignias

 Guardián de Secretos (10 pts) ·  Sembrador Reproducible (10 pts)
 Auditor de Fugas (15 pts) — pendiente del intercambio con un compañero

Resumen de ejecución

La Sesión 6 cierra el último punto pendiente antes del despliegue: el esquema de la base de datos se aplica solo en el pipeline, y ninguna credencial queda escrita en el repositorio.

Laboratorio	Nombre	Tiempo	Estado
1	Mover variables a GitHub Secrets	30 min	Cumplido
2	Migraciones en el pipeline	40 min	Cumplido
3	Auditoría de Fugas	45 min	Cumplido

Adaptación principal: el proyecto no usa Prisma

La guía asume Prisma para migraciones y seeds. Este proyecto no lo usa, así que se construyó el equivalente funcional: migraciones versionadas en SQL con su propia tabla de control. No es un atajo — reproduce las tres propiedades que hacen útil a ``prisma migrate deploy``: orden determinista, registro de lo ya aplicado y ejecución transaccional.

Guía (Prisma)	Este proyecto
<code>prisma migrate deploy</code>	<code>npm run db:migrate</code> — aplica los <code>.sql</code> pendientes en orden
<code>prisma db seed con upsert</code>	<code>npm run db:seed</code> — <code>INSERT ... ON CONFLICT DO UPDATE</code>
Tabla <code>_prisma_migrations</code>	Tabla <code>migracion_aplicada</code>
<code>prisma/migrations/</code>	<code>db/migraciones/001_...sql, 002_...sql</code>

Archivos creados

<code>db/migraciones/001_crear_tabla_tarea.sql</code>	Esquema de la tabla tarea
<code>db/migraciones/002_indice_completada.sql</code>	Indice para filtrar pendientes
<code>src/server/migraciones.js</code>	Aplicador de migraciones
<code>src/server/semilla.js</code>	Datos de ejemplo idempotentes
<code>src/server/cli.js</code>	Comandos <code>db:migrate</code> y <code>db:seed</code>
<code>src/server/migraciones.test.js</code>	11 pruebas de migraciones y seed

Además, `repositorio.inicializar()` dejó de crear la tabla con un `CREATE TABLE` embebido: ahora delega en las migraciones. El esquema tiene un único dueño, versionado en Git.

Laboratorio 1 — Mover variables a GitHub Secrets

Objetivo: identificar las variables sensibles, moverlas a GitHub Actions Secrets y consumirlas en el pipeline sin exponer nunca su valor.

Clasificación de las variables del proyecto

Variable	¿Sensible?	Por qué
DATABASE_URL	Sí	Contiene usuario y contraseña de la base de datos
JWT_SECRET	Sí	Firmaría las sesiones de los usuarios
API_KEY	Sí	Credencial de un servicio externo
PORT	No	Solo un número de puerto: no da acceso a nada
PUBLIC_API_URL	No	URL pública; el navegador la descarga de todos modos

Una precisión honesta sobre este proyecto: todavía no existe una base de datos en la nube ni autenticación, así que no hay credenciales reales que proteger. Los Secrets se crearon con valores deliberadamente ficticios para ejercitar el mecanismo; deberán reemplazarse por los reales cuando exista un entorno de despliegue.

Secrets creados

Secret	Valor cargado (ficticio)
DATABASE_URL	postgresql://ci_user:VALOR_FICTICIO_CAMBIAR@db.ejemplo.invalid:5432/appdb_staging
JWT_SECRET	jwt_de_practica_FALSO_cambiar_antes_de_produccion
API_KEY	sk_test_ROTADA_TRAS_LA_FUGA_valor_ficticio

El dominio .invalid está reservado por norma y no resuelve nunca: la cadena no apunta a ninguna base real.

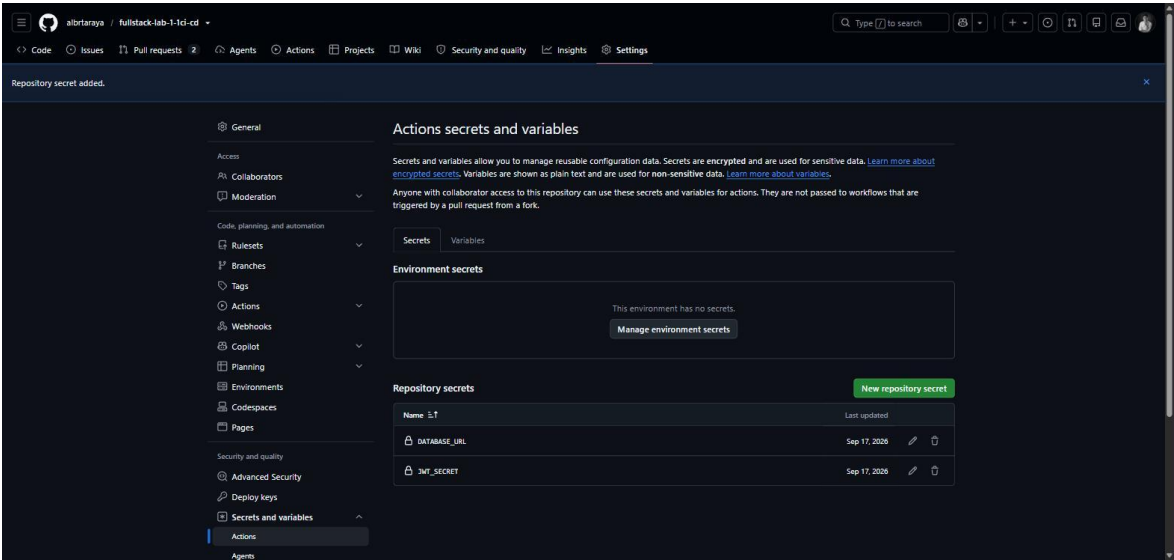


Figura 1. Settings → Secrets and variables → Actions. GitHub guarda los valores cifrados: ni el dueño del repositorio puede volver a leerlos, solo reemplazarlos.

Consumo en el workflow

```
pruebas:
  name: Pruebas unitarias y cobertura
  runs-on: ubuntu-latest
  env:
    JWT_SECRET: ${ secrets.JWT_SECRET }
```

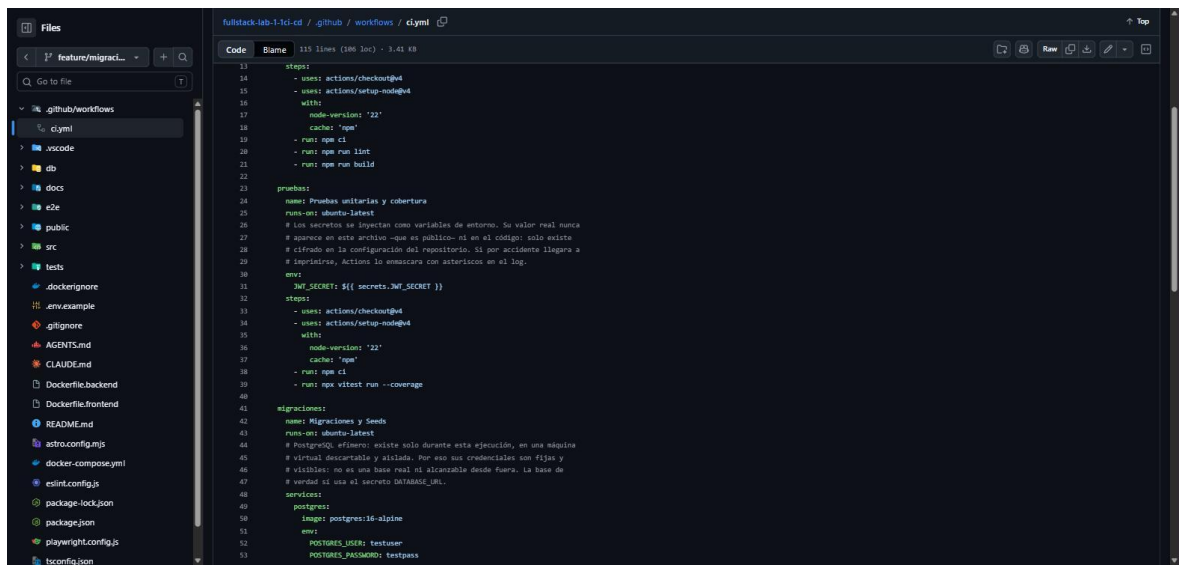


Figura 2. El ci.yml es público y solo contiene la referencia `${ secrets.JWT_SECRET }`: el valor no aparece por ninguna parte.

Verificación de que nada quedó expuesto

Se abrió el log del job que usa los secretos y se buscaron los fragmentos característicos de cada valor cargado:

Fragmento buscado en el log	Resultado
jwt_de_practica	No aparece
VALOR_FICTICIO	No aparece
ci_user	No aparece
sk_test_ROTADA	No aparece

Dos mecanismos actúan en conjunto: el valor nunca se escribe en el repositorio, y si por accidente llega a imprimirse, Actions lo sustituye por asteriscos en el log.

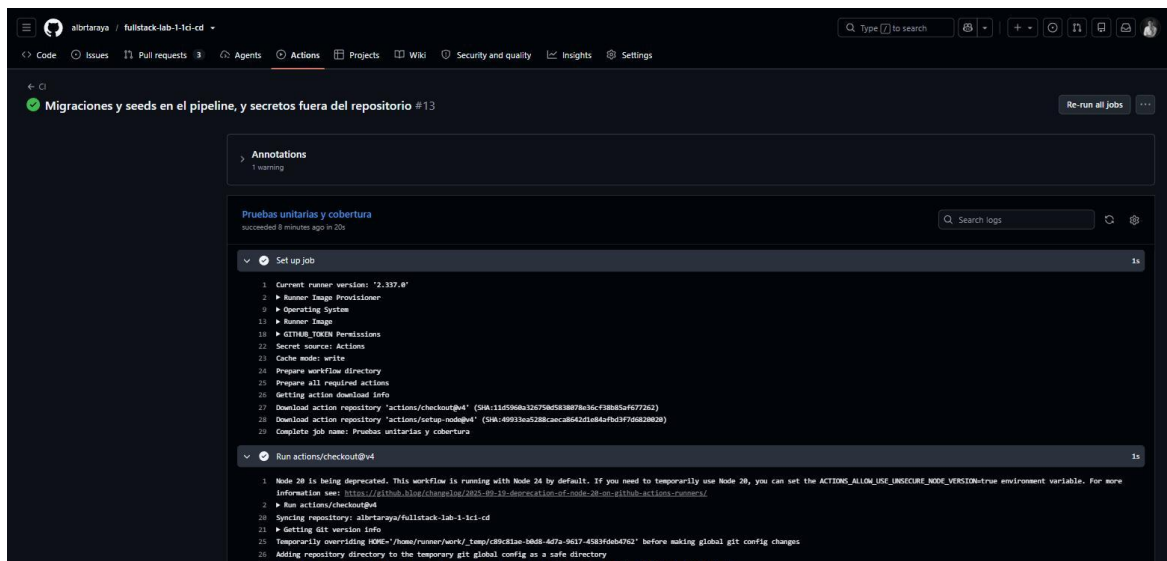


Figura 3. Log del job «Pruebas unitarias y cobertura». En «Set up job» se lee «Secret source: Actions», sin ningún valor en texto plano.

Checkpoint del Laboratorio 1

- ✓ Secrets creados en Settings → Secrets and variables → Actions.
- ✓ El ci.yml los consume con la sintaxis `${{ secrets.NOMBRE }}`.
- ✓ Se verificó en los logs que ningún valor real quedó expuesto.

Laboratorio 2 — Migraciones en el pipeline

Objetivo: agregar un PostgreSQL efímero al pipeline y un paso que aplique las migraciones automáticamente contra esa base de prueba.

El servicio efímero

```
migraciones:
  name: Migraciones y Seeds
  runs-on: ubuntu-latest
  services:
    postgres:
      image: postgres:16-alpine
      env:
        POSTGRES_USER: testuser
        POSTGRES_PASSWORD: testpass
        POSTGRES_DB: testdb
      ports:
        - 5432:5432
      options: >-
        --health-cmd pg_isready
        --health-interval 5s
        --health-timeout 5s
        --health-retries 5
      env:
        DATABASE_URL: postgresql://testuser:testpass@localhost:5432/testdb
```

Por qué aquí las credenciales sí van a la vista: este PostgreSQL existe solo durante los segundos que dura la ejecución, en una máquina virtual descartable y aislada de GitHub. No es una base real ni alcanzable desde fuera. La base de verdad sí usa el Secret del Laboratorio 1.

El aplicador de migraciones

Cada migración corre dentro de su propia transacción, junto con su registro en la tabla de control: si el SQL falla, se hace ROLLBACK y la migración no queda marcada como aplicada. Eso evita el peor escenario — una migración a medias que el sistema cree completa.

```
await pool.query('BEGIN');
try {
  await pool.query(sql);
  await pool.query('INSERT INTO migracion_aplicada (nombre) VALUES ($1)', [nombre]);
  await pool.query('COMMIT');
  aplicadas.push(nombre);
} catch (error) {
  await pool.query('ROLLBACK');
  throw new Error(`Fallo la migracion ${nombre}: ${error.message}`, { cause: error });
}
```

Resultado en el pipeline

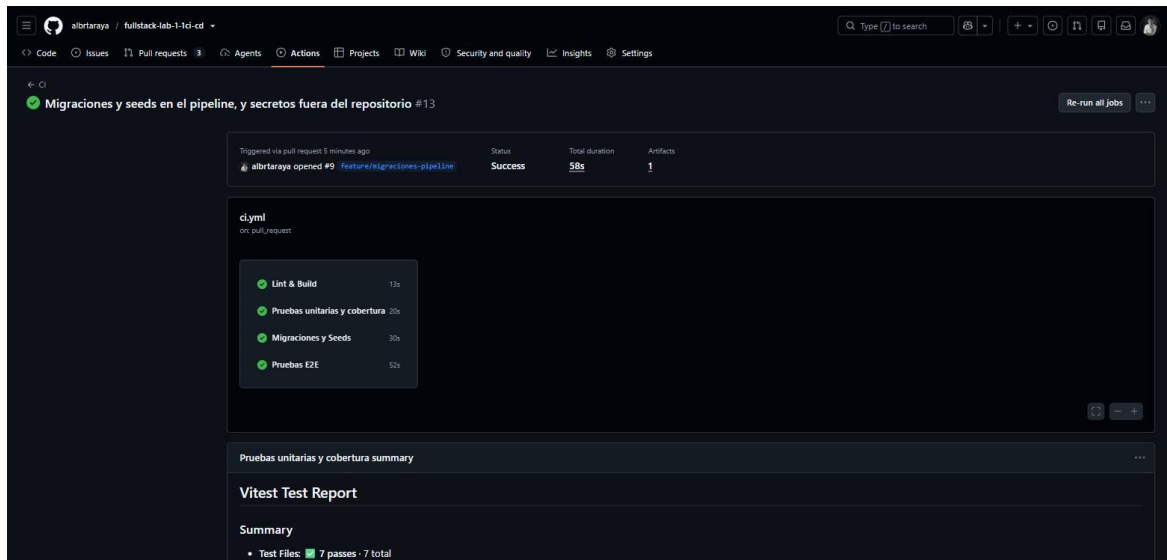


Figura 4. El pipeline pasa de tres a cuatro jobs. «Migraciones y Seeds» completa en 30 s, con «Initialize containers» levantando el PostgreSQL efímero.

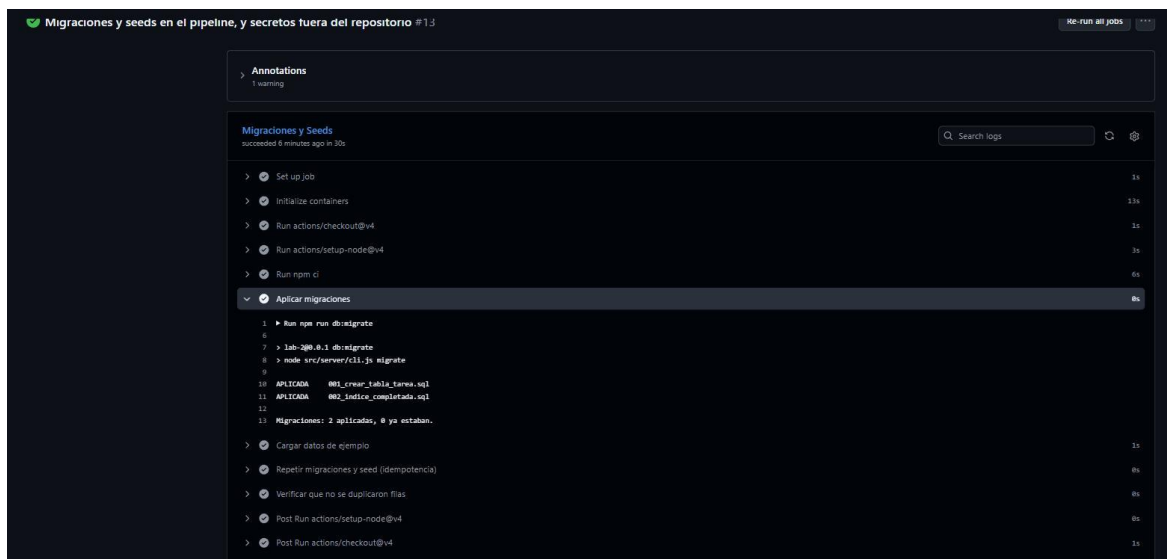


Figura 5. Primera corrida en CI: las dos migraciones se aplican sobre una base recién creada.

Checkpoint del Laboratorio 2

- ✓ El servicio de PostgreSQL efímero está declarado con healthcheck en el ci.yml.
- ✓ El paso de migraciones corre y termina en verde.
- ✓ Queda claro por qué esta base de prueba no necesita Secrets y la de producción sí.

Laboratorio 3 — Auditoría de Fugas

Objetivo: practicar, con valores completamente ficticios, cómo se detecta y se remedia una fuga de credenciales, tanto en el código actual como en el historial.

Regla de seguridad respetada: todos los valores usados son obviamente falsos (sk_test_FALSO123456789, contraseñaDePrueba999). No se empleó ninguna credencial real, ni siquiera de un entorno de prueba verdadero.

El seed reproducible

El equivalente al upsert de Prisma en SQL plano es ON CONFLICT DO UPDATE: si la fila ya existe, la actualiza en lugar de fallar o duplicarla.

```
await pool.query(`
  INSERT INTO tarea (id, titulo, completada)
  VALUES ($1, $2, $3)
  ON CONFLICT (id) DO UPDATE
  SET titulo = EXCLUDED.titulo, completada = EXCLUDED.completada`,
  [id, titulo, completada],
);
```

Un detalle que suele morder: tras insertar con ids explícitos hay que reajustar la secuencia con `setval('tarea_id_seq', ...)`. Sin eso, el siguiente alta automática intentaría reutilizar un id ya ocupado y fallaría por clave duplicada. Se verificó creando una tarea desde la API después del seed: recibió el id 4, correctamente.

Demostración de idempotencia

Local, contra el PostgreSQL de Docker Compose:

```
$ npm run db:migrate          # 1ra vez
APLICADA    001_crear_tabla_tarea.sql
APLICADA    002_indice_completada.sql
Migraciones: 2 aplicadas, 0 ya estaban.

$ npm run db:migrate          # 2da vez
ya aplicada 001_crear_tabla_tarea.sql
ya aplicada 002_indice_completada.sql
Migraciones: 0 aplicadas, 2 ya estaban.

$ npm run db:seed             # 1ra vez
Total de filas en la tabla tarea: 3

$ npm run db:seed             # 2da vez
Total de filas en la tabla tarea: 3
```

Y el pipeline lo verifica solo en cada Pull Request: repite migrate + seed y falla si el conteo de filas deja de ser 3.

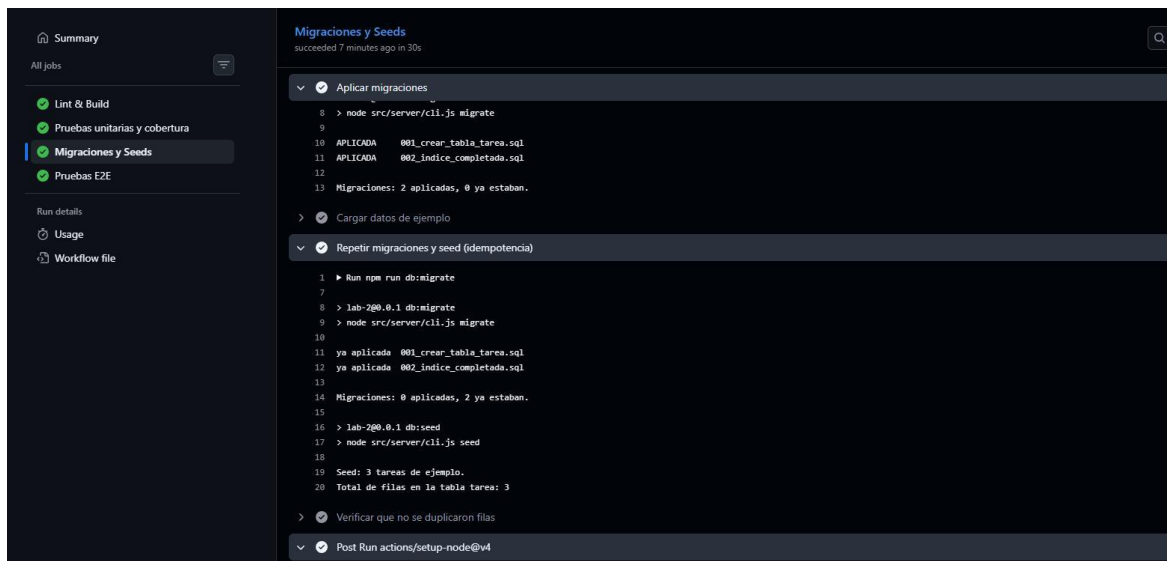


Figura 6. Segunda corrida en CI: «0 aplicadas, 2 ya estaban» y «Total de filas: 3». El seed no duplicó nada.

Insignia obtenida:  Sembrador Reproducible (10 puntos)

Etaa 1 — Plantar las fugas

En la rama de práctica feature/auditoria-fugas se plantaron dos fugas de distinta naturaleza:

Fuga	Dónde	Qué se plantó
1 — código actual	src/server/configuracion.js	const apiKey = 'sk_test_FALSO123456789'
2 — historial	.env.filtrado	Una cadena de conexión con contraseña, forzada con git add -f

La segunda se eliminó en un commit posterior con git rm, pero sin limpiar el historial: precisamente para comprobar que eso no basta.

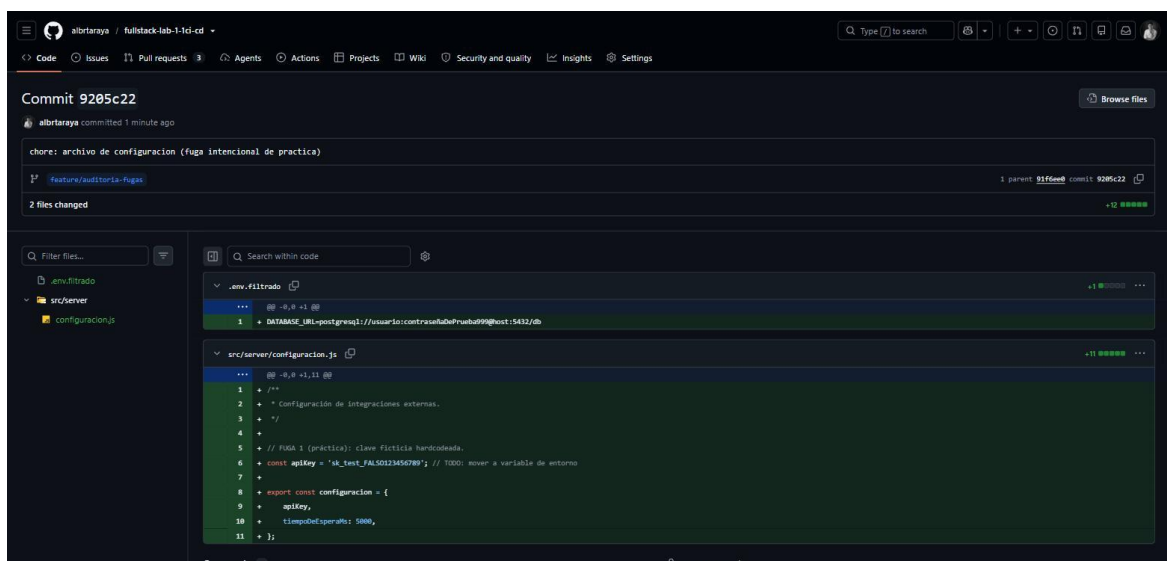


Figura 7. Commit 9205c22 con ambas fugas: la cadena de conexión y la clave hardcodeada.

Etapla 2 — La auditoría

Fuga 1, con grep sobre el código actual:

```
$ grep -rn "apiKey\\|API_KEY\\|SECRET\\|sk_test\\|sk_live" --include="*.js" src/  
src/server/configuracion.js:6:const apiKey = 'sk_test_FALS0123456789'; // TODO: mover...
```

Fuga 2, revisando el historial completo:

```
$ git log --all --oneline  
10eb3d1 chore: eliminar archivo de configuracion  
9205c22 chore: archivo de configuracion (fuga intencional de practica)  
  
$ git log -p --all -- .env.filtrado  
commit 9205c22...  
+DATABASE_URL=postgresql://usuario:contraseñaDePrueba999@host:5432/db
```

Fuga	Archivo	Commit
1 — clave hardcodeada	src/server/configuracion.js, línea 6	9205c22
2 — credencial en el historial	.env.filtrado	9205c22 (añadida) · 10eb3d1 (borrada)

La lección: borrar el archivo no elimina la fuga

El archivo ya no existe en el último estado de la rama, y aun así GitHub lo sigue sirviendo íntegro si se pide por el hash del commit donde estuvo:

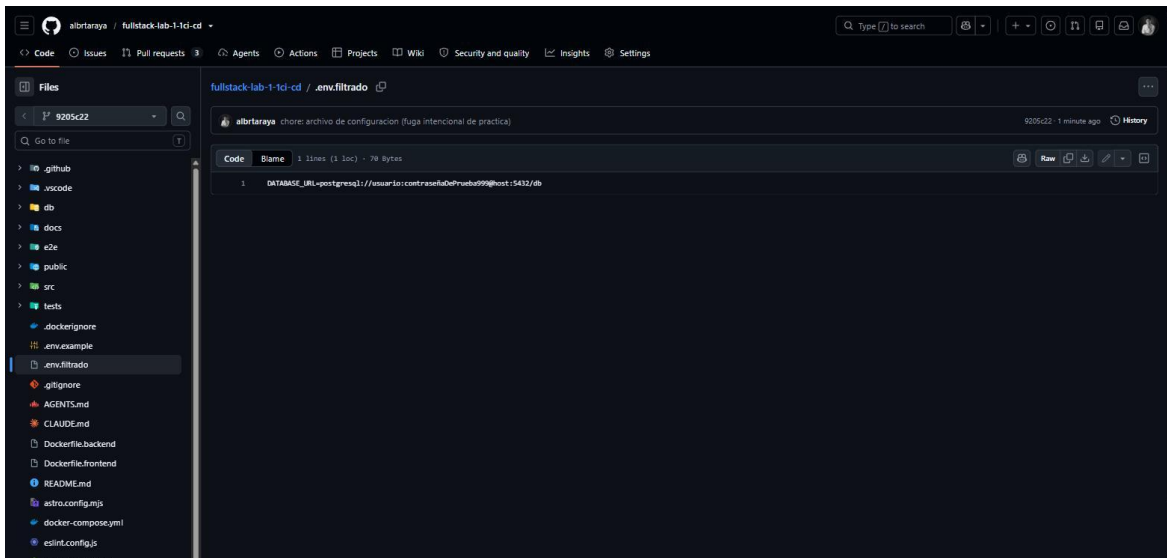


Figura 8. .env.filtrado, ya eliminado de la rama, sigue accesible y legible en /blob/9205c22/.env.filtrado.

Etapla 3 — Remediación

Fuga 1: el valor sale del código y pasa a una variable de entorno respaldada por un Secret.

```
// Antes
const apiKey = 'sk_test_FALSO123456789'; // TODO: mover a variable de entorno

// Después
const apiKey = process.env.API_KEY;

$ grep -rn "sk_test|sk_live" --include="*.js" src/
(sin coincidencias: fuga remediada)
```

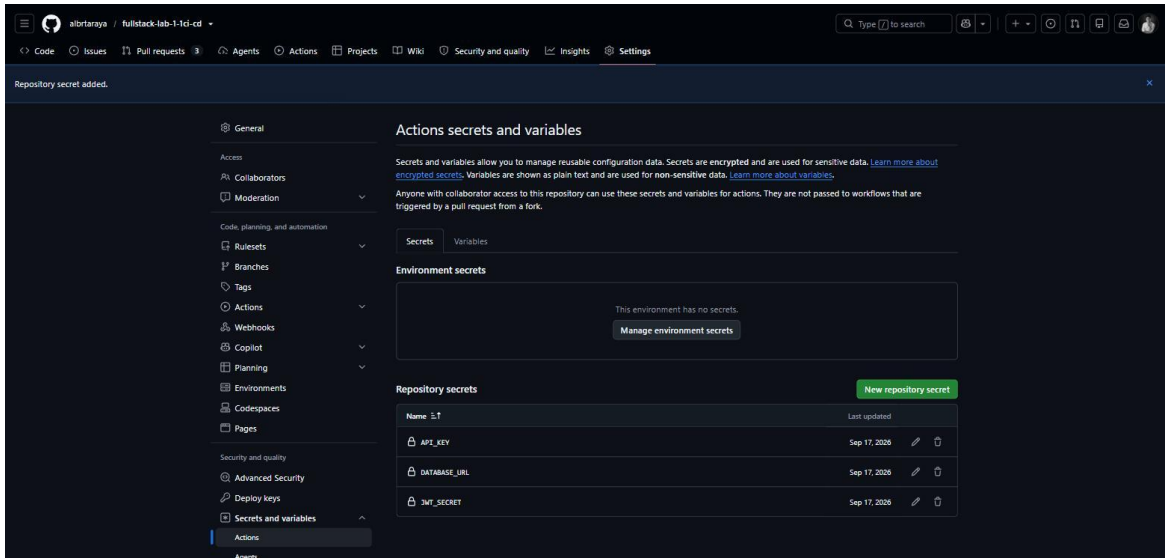


Figura 9. El Secret `API_KEY` creado como parte de la remediación, junto a los otros dos.

Fuga 2: qué correspondería en un caso real

Borrar el archivo en un commit posterior NO es remediación. La credencial sigue siendo recuperable por cualquiera con acceso al repositorio, y también por quien haya clonado antes. La respuesta profesional tiene dos partes, en este orden:

1. ROTAR la credencial — generar una nueva e invalidar la anterior. Esto es lo urgente: mientras la clave siga siendo válida, da igual cuán bien se limpie el historial, porque ya pudo ser copiada.
2. Limpiar el historial con `git filter-repo` o `BFG Repo-Cleaner`, reescribiendo los commits afectados. Queda fuera del alcance de este laboratorio.

Cierre del ejercicio

La rama de práctica no se fusionó a main y se eliminó en ambos lados, tal como indica la guía:

```
$ git branch -D feature/auditoria-fugas
Deleted branch feature/auditoria-fugas (was 0e6704a).

$ git push origin --delete feature/auditoria-fugas
- [deleted]          feature/auditoria-fugas
```

Verificación de que nada de la práctica llegó a main:

```
$ git ls-tree -r --name-only main | grep env.filtrado
(ausente)

$ git grep -n "sk_test_FALSO" main
```

```
(ausente)
```

```
$ git ls-tree -r --name-only main | grep configuracion.js  
(ausente)
```

Insignia obtenida:  **Guardián de Secretos (10 puntos)**

Insignia pendiente:  **Auditor de Fugas (15 puntos)** — requiere auditar las fugas de un compañero

Alcance de esta parte

La actividad se hizo de forma individual. Las fugas se plantaron, se auditaron con los comandos que indica la guía y se remediaron, pero el intercambio con una pareja — encontrar las fugas de otra persona sin pistas — no pudo realizarse. Esa es la condición de la insignia «Auditor de Fugas», que queda pendiente.

Entregable final de la Sesión 6

Requisito	Estado
Las variables sensibles están en GitHub Secrets, ninguna en texto plano	Cumplido
El pipeline aplica las migraciones contra una base de prueba efímera	Cumplido
El seed corre de forma reproducible, sin duplicar datos	Cumplido
Se completó el ejercicio de Auditoría de Fugas, encontrando y remediando ambas	Cumplido
La rama feature/auditoria-fugas fue eliminada al terminar	Cumplido

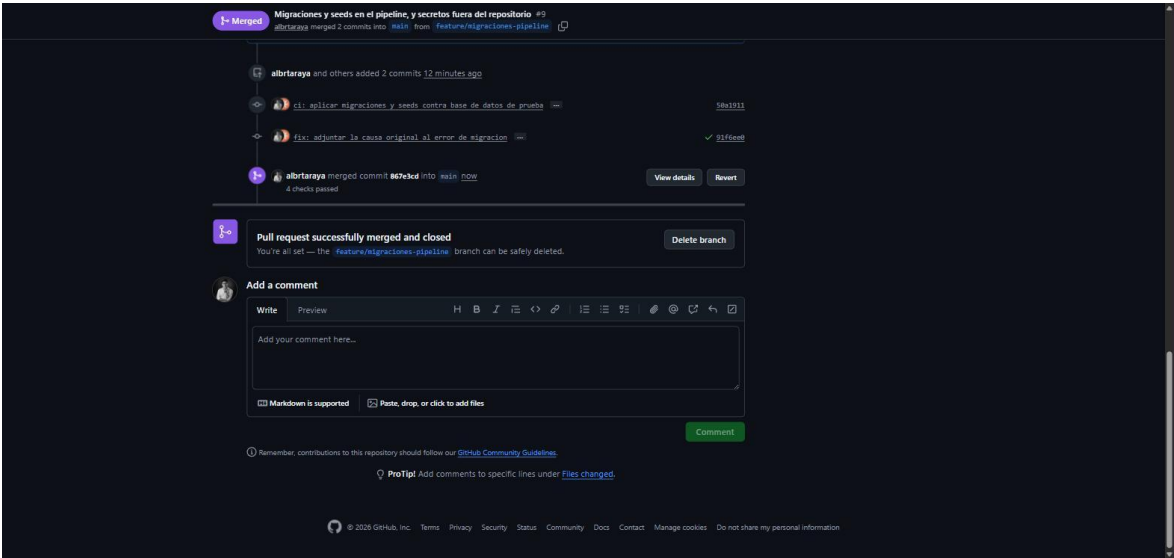


Figura 10. Pull Request #9 integrado a main en el commit 867e3cd, con los 4 checks en verde.

Estado del pipeline tras esta sesión

Job	Qué verifica	Duración
Lint & Build	ESLint y build de Astro	13s
Pruebas unitarias y cobertura	Vitest con umbrales de cobertura	20s
Migraciones y Seeds	Migraciones, seed e idempotencia contra PostgreSQL	30s
Pruebas E2E	Playwright sobre el navegador real	52s

Pendientes declarados

Pendiente	Qué falta
Valores reales en los Secrets	Los tres Secrets tienen valores ficticios: no existe aún base en la nube ni autenticación. Reemplazarlos al desplegar
Insignia Auditor de Fugas	Requiere auditar las fugas plantadas por un compañero
Revisión por pares en los PRs	El repositorio no tiene colaboradores; los checks automáticos son el único guardián

Siguiente estación

Con el código probado, contenerizado, con su base de datos migrada y sus secretos bajo control, la Sesión 7 cierra el círculo: el pipeline desplegará la aplicación a la nube automáticamente, con ambientes de staging y producción.

Anexo A — Índice de evidencias

Archivo	Contenido
01-secrets-creados.jpg	Los tres Secrets en la configuración del repositorio
02-ciyaml-sintaxis-secrets.jpg	ci.yml público con <code>\${{ secrets.JWT_SECRET }}</code>
03-log-sin-valores-expuestos.jpg	Log del job sin valores en texto plano
04-run-cuatro-jobs-verde.jpg	Los cuatro jobs del pipeline en verde
05-migraciones-aplicadas-en-ci.jpg	Primera corrida: migraciones aplicadas
06-idempotencia-segunda-corrida.jpg	Segunda corrida: 0 aplicadas, 3 filas
07-fugas-plantadas-commit.jpg	Commit con las dos fugas ficticias
08-archivo-borrado-sigue-accesible.jpg	El archivo borrado sigue siendo legible
09-secret-apikey-tras-remediar.jpg	Secret API_KEY creado en la remediación
10-pr9-merged.jpg	Pull Request #9 integrado a main

Anexo B — Pruebas agregadas

El job de cobertura exige que el código nuevo esté probado, así que las migraciones y el seed se cubrieron con 11 pruebas que simulan el driver de PostgreSQL:

Aspecto verificado	Por qué importa
Orden por prefijo numérico	Una migración fuera de orden rompe el esquema
No repetir lo ya aplicado	Es lo que hace seguro correr el comando dos veces
Registro en la tabla de control	Sin registro, todo se reaplicaría siempre
BEGIN / COMMIT por migración	Aísla cada cambio de esquema
ROLLBACK ante fallo, sin marcarla	Evita una migración a medias dada por buena
ON CONFLICT DO UPDATE en el seed	Es la cláusula que lo hace repetible
setval tras el seed	Evita el choque de ids en el siguiente alta

```
$ npx vitest run src/server tests
Test Files  4 passed (4)
Tests      33 passed (33)
```

Anexo C — Commits de la Sesión 6

```
50a1911 ci: aplicar migraciones y seeds contra base de datos de prueba
91f6ee0 fix: adjuntar la causa original al error de migracion
867e3cd Merge pull request #9 from albrtaraya/feature/migraciones-pipeline

Rama de practica (eliminada al terminar, nunca fusionada):
9205c22 chore: archivo de configuracion (fuga intencional de practica)
10eb3d1 chore: eliminar archivo de configuracion
```